universität wien

MULTIOMICS DATA SCIENCE – CLASSIFICATION

CLASSIFICATION

- Method (model) for projection of observed / measured data (features) on a target variable → Question: is there any correlation between the features and the target variable

- Observations with known target variable are called labelled data

- Target variable must be nominell (z.b. red / yellow / green), otherwise when dealing with a numerical target variable we speak about *regressions*

CLASSIFICATION

- Method (model) for projection of observed / measured data (features) on a target variable → Question: is there any correlation between the features and the target variable
- Observations with known target variable are called labelled data
- Target variable must be nominell (z.b. red / yellow / green), otherwise when dealing with a numerical target variable we speak about *regressions*
- Target variable shoud have a finate set of values / groups / classes
- With a trainied model, unlabelled data can be assigned to a class (**Prediction**)

WHICH CLASSIFICATORS

- Decision Tree
- Random Forest
- Artificial Neuronal Network
- Naive Bayes Model
- Support Vector Machine (SVM)

INPUT DATA

- All classification implementations work on numerical data
- Nominal data need to be converted to numerical data

Age	Experience	Rank	Nationality	Go
36	10	9	UK	NO
42	12	4	USA	NO
23	4	6	N	NO
52	4	4	USA	NO
43	21	8	USA	YES
44	14	5	UK	NO
66	3	7	N	YES

Quelle: *https://www.w3schools.com/python/python_ml_decision_tree.asp*

INPUT DATA

Age	Experience	Rank	~~Nationality~~	Go	Nationality_UK	Nationality_USA	Nationality_N
36	10	9	~~UK~~	NO	1	0	0
42	12	4	~~USA~~	NO	0	1	0
23	4	6	~~N~~	NO	0	0	1
52	4	4	~~USA~~	NO	0	1	0
43	21	8	~~USA~~	YES	0	1	0

Quelle: *https://www.w3schools.com/python/python_ml_decision_tree.asp*

INPUT DATA

Age	Experience	Rank	~~Nationality~~	Go
36	10	9	~~UK~~	NO
42	12	4	~~USA~~	NO
23	4	6	~~N~~	NO
52	4	4	~~USA~~	NO
43	21	8	~~USA~~	YES

Nationality_UK	Nationality_USA	Nationality_N
1	0	0
0	1	0
0	0	1
0	1	0
0	1	0

Quelle: *https://www.w3schools.com/python/python_ml_decision_tree.asp*

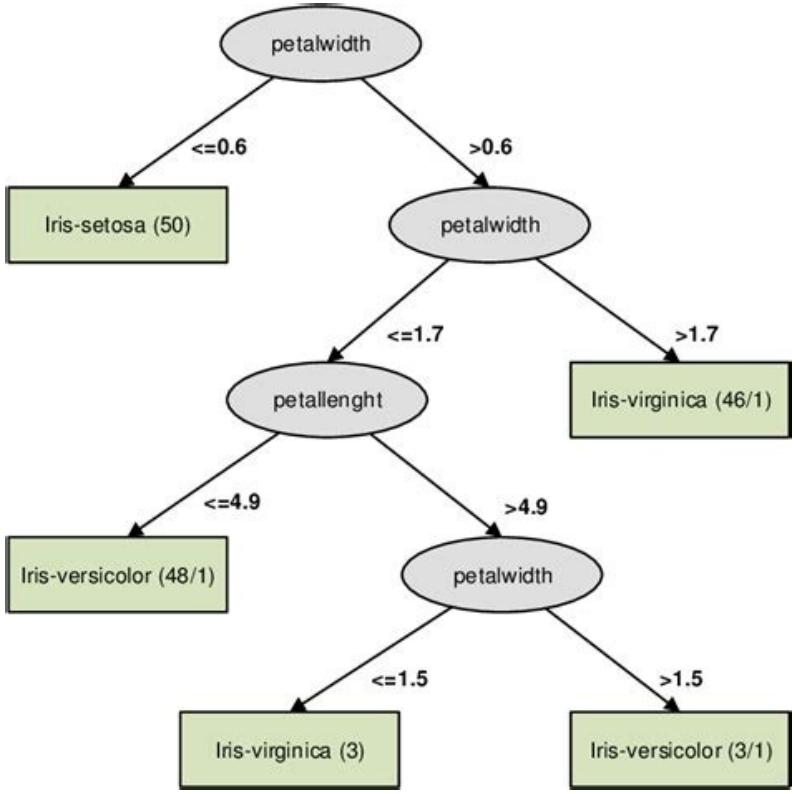
```
# df is data frame with all data
nationalities = set(df['Nationality'])
for natio in nationalities:
    df['Nationality_' + natio] = 1 if df['Nationality'] == natio else 0
df = df.drop(['Nationality'], axis = 1)
```

universität wien

DECISION TREE

- Most intuitive model
- On numerical values: regression tree; on nominal values: classification tree
- Model matches a tree structure with:
  - Features in the nodes
  - Thresholds of the feature values in the edges
  - Unique class in the leafs

DECISION TREE – IRIS EXAMPLE



Quelle: *https://www.researchgate.net/figure/Decision-tree-for-Iris-dataset_fig1_293194222*

DECISION TREE

- Parameters for decision tree:
  - `criterion`: function to determine the „impurity“ of the nodes {“`gini`“, “`entropy`“, “`log_loss`“}
  - `max_depth`: maximum depth of the tree
  - `min_samples_split`: Minimal number of observations, where an internal node may be split
  - `random_state`: Determine of a start value for the internal random generator

RANDOM FOREST

- The random forest (RF) is an extension of the decision tree (DT)
- Instead of one DT, several DTs are created
- During creation of the DT, only a subset of features is used

RANDOM FOREST

- The random forest (RF) is an extension of the decision tree (DT)
- Instead of one DT, several DTs are created
- During creation of the DT, only a subset of features is used
- When applying the RF, all DTs are used
- The majority of all DT classifications is used as prediction

RANDOM FOREST

- Parameters for random rorest:
  - `criterion`: function to determine the „impurity“ of the nodes {“`gini`“, “`entropy`“, “`log_loss`“}
  - `max_depth`: maximum depth of the tree
  - `min_samples_split`: Minimal number of observations, where an internal node may be split
  - `random_state`: Determine of a start value for the internal random generator
  - `n_estimators`: number of trees in the forest

ARTIFICIAL NEURONAL NETWORK

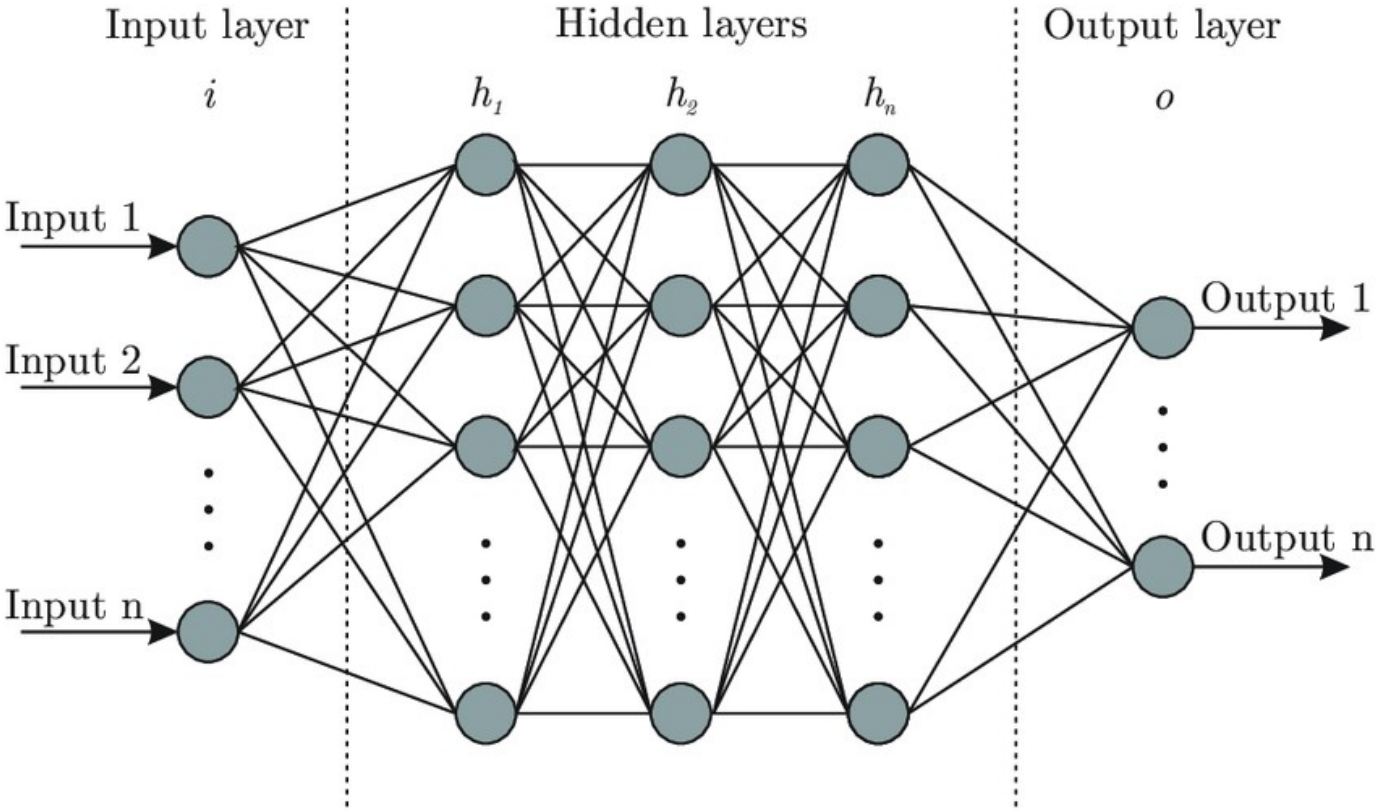
- Network consisting of nodes and edges, behavior of nodes immitates neurons in the brain

- Iterative algorithm for model training

ARTIFICIAL NEURONAL NETWORK

- Network consisting of nodes and edges, behavior of nodes immitates neurons in the brain
- Iterative algorithm for model training
- Three neuron layers exist:
  - Input layer, processing features
  - Output layer, determining the class
  - Hidden layer, connecting both outer layers
- Three-layer model is rigid, the number of nodes is variable, and edge weights are estimated during the training

ARTIFICIAL NEURONAL NETWORK



Quelle: *https://towardsdatascience.com/designing-your-neural-networks-a5e4617027ed*

ARTIFICIAL NEURONAL NETWORK

- Parameters for neuronal networks:
  - `hidden_layer_sizes`: Number of neurons per hidden layer, default: (100,)
  - `activation`: activation function of the neurons {“`identity`“, “`logistic`“, “`tanh`“, “`relu`“}
  - `max_iter`: number of repetitions for the training, default: 200
  - `tol`: minimal difference of the quality function between two training loops for termintaion, default: 10^{-4}
  - `random_state`: determine of a start value for the internal random generator

DEEP LEARNING

- Classic artificial neuronal networks have three layers
- „Deep Learning“ referres to artificial networks, with multiple hidden layers

DEEP LEARNING

- Classic artificial neuronal networks have three layers
- „Deep Learning“ referres to artificial networks, with multiple hidden layers
- Advantage:
  - Better abstraction between the input layer and output layer
- Disadvantage:
  - Many, many, many trainging data necessary
  - Method prone to *overfitting*

NAIVE BAYES

- Based on the Bayes theorem : $p(A|B)=\frac{p(B|A)\cdot p(A)}{p(B)}$
- Only non-parametrical classifier
- On numerical data, a normal distribution of feature values is assumed, the parameters μ, σ are estimated for each feature
- The method is called naive, because it assumes complete independence of all parameters

NAIVE BAYES

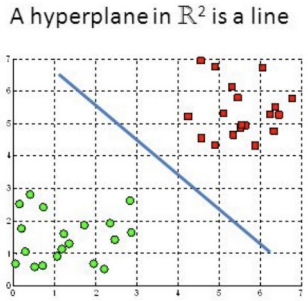
- Based on the Bayes theorem : $p(A|B)=\frac{p(B|A)\cdot p(A)}{p(B)}$
- Only non-parametrical classifier
- On numerical data, a normal distribution of feature values is assumed, the parameters μ, σ are estimated for each feature
- The method is called naive, because it assumes complete independence of all parameters
- For prediction, a likelihood is computed for each class
- The class with highest likelihood wins

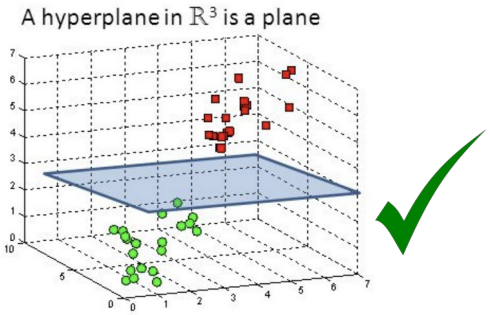
SUPPORT VECTOR MACHINE

- Method puts **hyper planes** in the feature space that separate the classes as good as possible
- Hyper planes is linear in each dimension in the standard model, but can be changed with other functions
- Iterative algorithm for model training

SUPPORT VECTOR MACHINE

- Method puts **hyper planes** in the feature space that separate the classes as good as possible
- Hyper planes is linear in each dimension in the standard model, but can be changed with other functions
- Iterative algorithm for model training





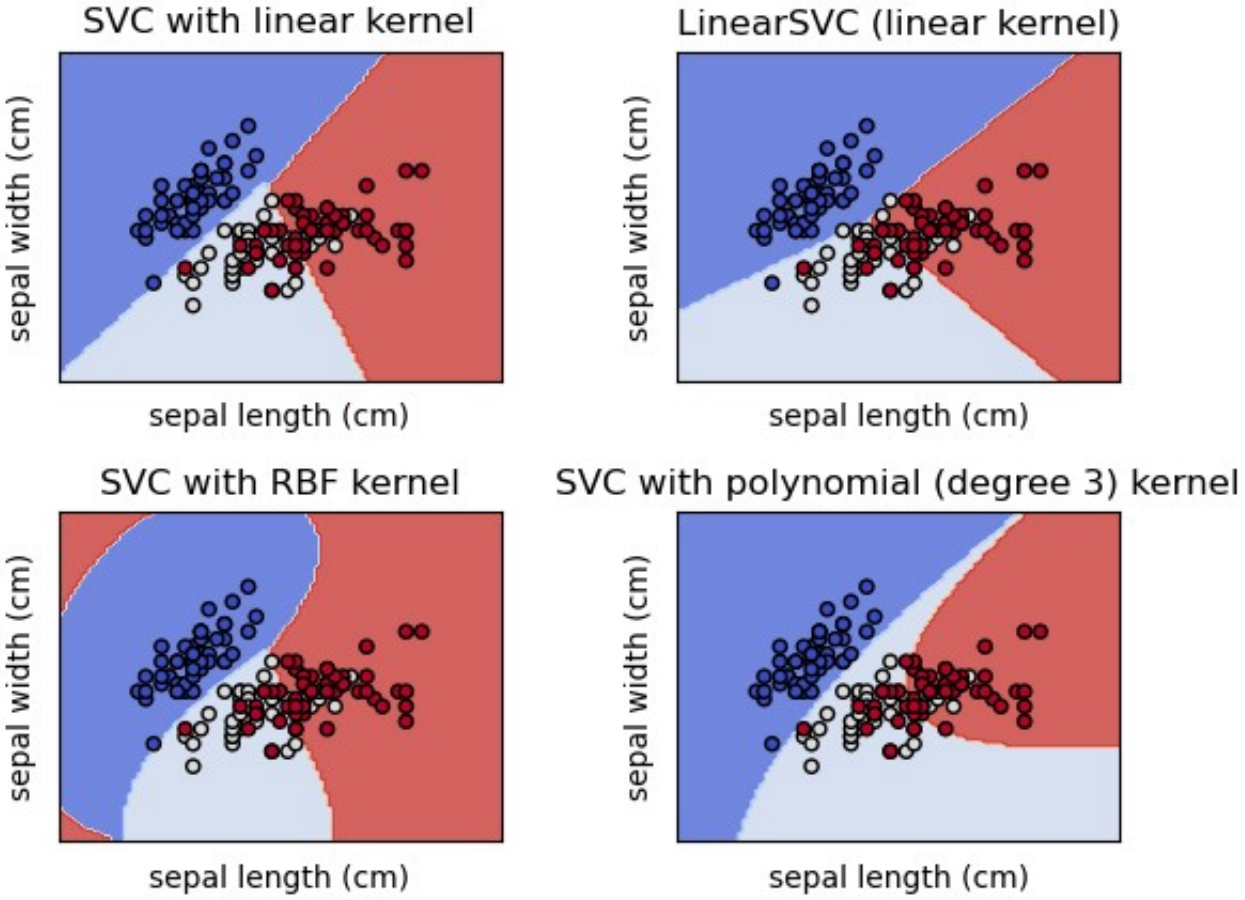
Source: *https://rifqimulyawan.com*

Source: *https://reddit.com*

SUPPORT VECTOR MACHINE

- Method puts **hyper planes** in the feature space that separate the classes as good as possible
- Hyper planes is linear in each dimension in the standard model, but can be changed with other functions
- Iterative algorithm for model training
- A hyper band is put between the classes as wide as possible
- These bands are described with support vectors

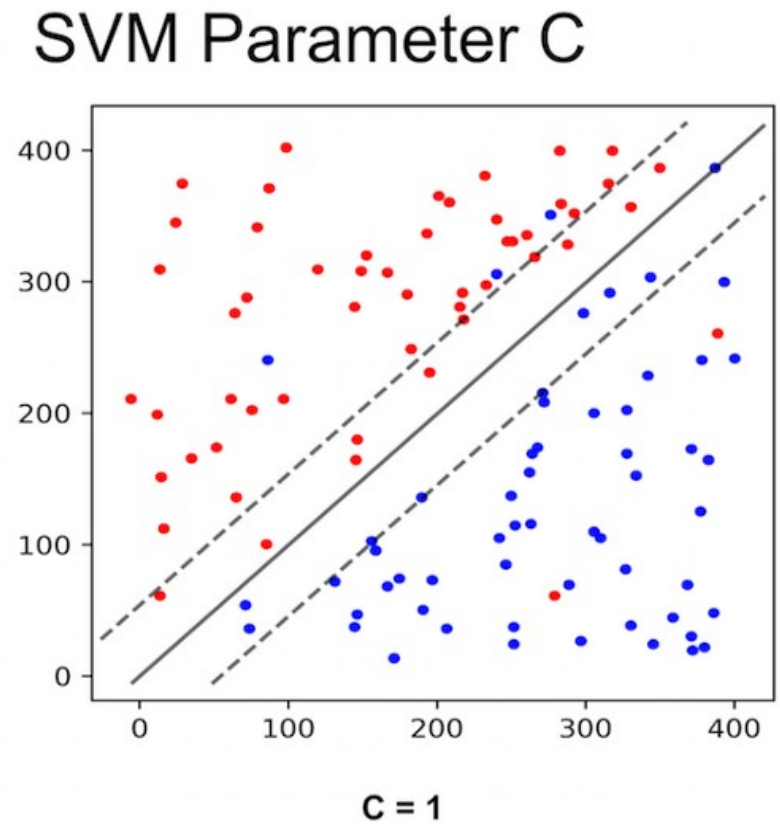
SUPPORT VECTOR MACHINE APPLIED ON IRIS DATASET

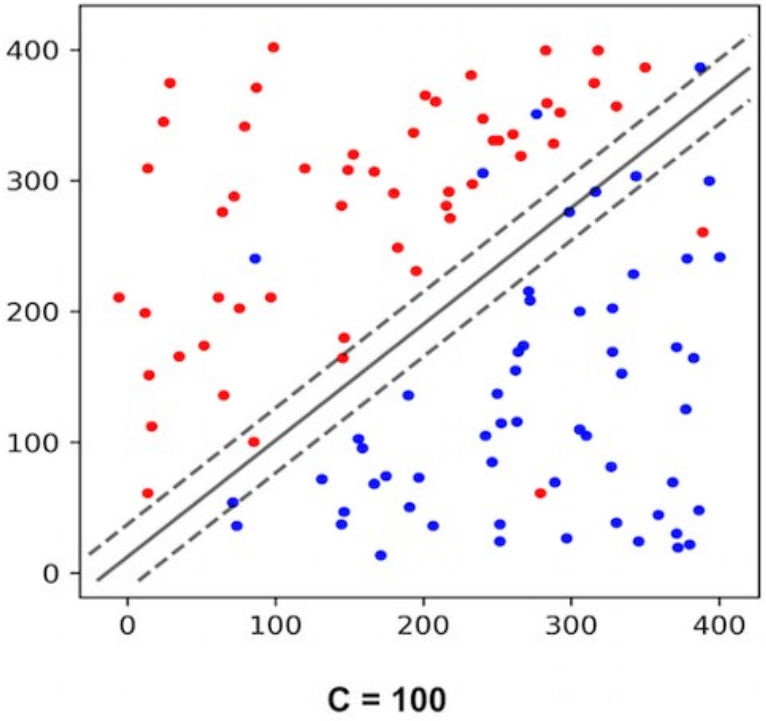


Quelle: *https://scikit-learn.org/stable/modules/svm.html*

universität wien

SUPPORT VECTOR MACHINE





Quelle: https://medium.com/@cdabakoglu/what-is-support-vector-machine-svm-fd0e9e39514f

universität wien

SUPPORT VECTOR MACHINE

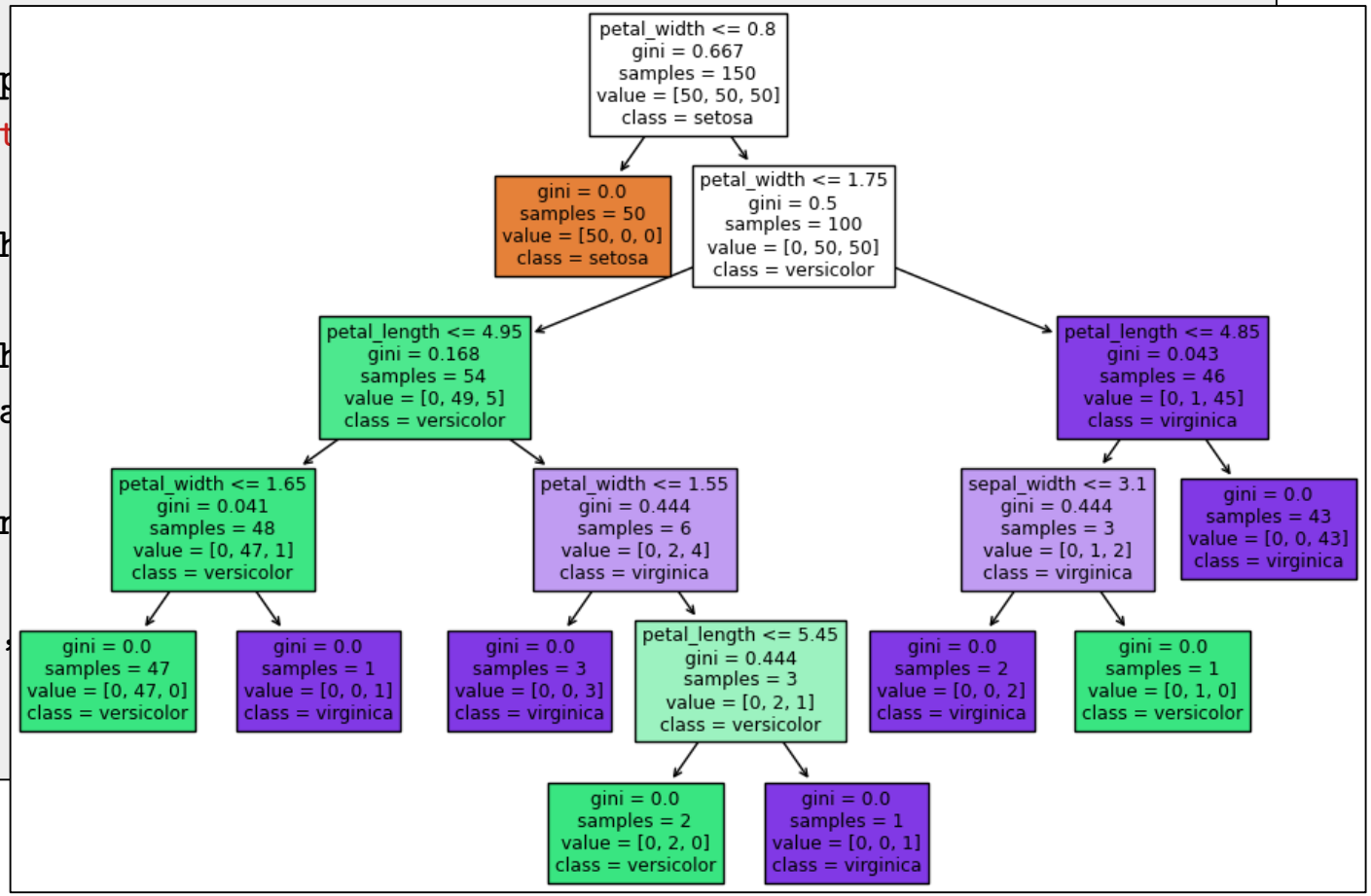
- Parameters for SVM:
  - `kernel`: Function for determination of the hyper plane {“`linear`“, “`poly`“, “`rbf`“, “`sigmoid`“}
  - `C`: width of the band; big C → small band, more precise but more prone to errors, default: 1.0
  - `max_iter`: maximal number of repetitions for training, default: 100
  - `tol`: minimal difference of the quality function between two training loops for termintaion, default: 10^{-4}
  - `random_state`: determine of a start value for the internal random generator

EXAMPLE WITH THE IRIS DATASET

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import tree

iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
feat = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
X, y = iris.loc[:,feat], iris["species"]

model = tree.DecisionTreeClassifier(random_state = 0)
model.fit(X, y)
tree.plot_tree(model, feature_names = feat, class_names = model.classes_,
filled = True)
plt.show()
```

EXAMPLE WITH THE IRIS DATASET

```
import pandas as pd
import matplotlib.pyp
from sklearn import t

iris = pd.read_csv('h
master/iris.csv')
feat = ["sepal_length
X, y = iris.loc[:,fea

model = tree.Decision
model.fit(X, y)
tree.plot_tree(model,
filled = True)
plt.show()
```



universität wien

EXAMPLE WITH ALL CLASSIFICATORS

```
from sklearn import tree
from sklearn import svm
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.naive_bayes import GaussianNB

#model = tree.DecisionTreeClassifier(random_state = 0)
model = svm.SVC(kernel = "linear", random_state = 0)
#model = RandomForestClassifier(random_state = 0)
#model = MLPClassifier(random_state = 0)
#model = GaussianNB()
model.fit(X, y)
```

DIMENSIONS OF THE DATASETS

- The more features a dataset contains and the more classes a target variable has, the more observations a dataset needs to have

- The problem of high-dimensional data is called *Curse of Dimensionality*

DIMENSIONS OF THE DATASETS

- The more features a dataset contains and the more classes a target variable has, the more observations a dataset needs to have

- The problem of high-dimensional data is called *Curse of Dimensionality*

- Beispiel:
  - 1000 features, 10 observations (5 in class *A*, 5 in class *B*)

  - Prob, that a feature splits both classes by chance is:
    $2 * 5! * 5! / 10! = 0.007936$

  - The expectation value for 1000 features is: $0.007936 * 1000 \approx 8$

DIMENSIONS OF THE DATASETS

- The more features a dataset contains and the more classes a target variable has, the more observations a dataset needs to have

- The problem of high-dimensional data is called *Curse of Dimensionality*

Rule of thumb: per class
5 x (# features) < # observations

2 * 5! * 5! / 10! = 0.007936

- The expectation value for 1000 features is: 0.007936 * 1000 ≈ 8

universität wien

UNDERFITTING / OVERFITTING

- Underfitting: classification to “wide“ due to too few features

- Examples:
  - Regression with linear model on non-linear data

  - “When it’s brown, it’s a tree“

UNDERFITTING / OVERFITTING

- Underfitting: classification to “wide“ due to too few features

- Examples:
  - Regression with linear model on non-linear data

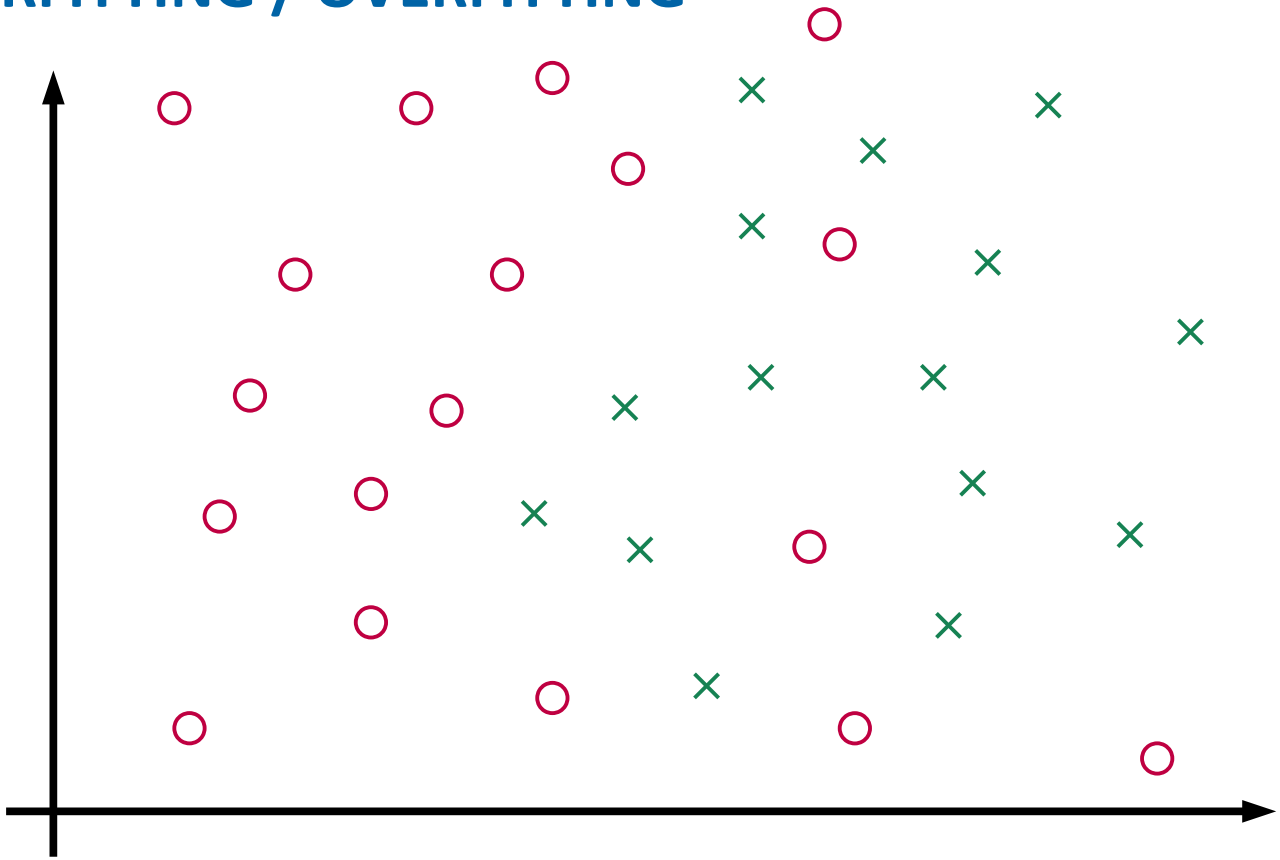
  - “When it’s brown, it’s a tree“

- Overfitting: classification to narrow due to to many features

- Examples:
  - Regression with non-linear model on linear data

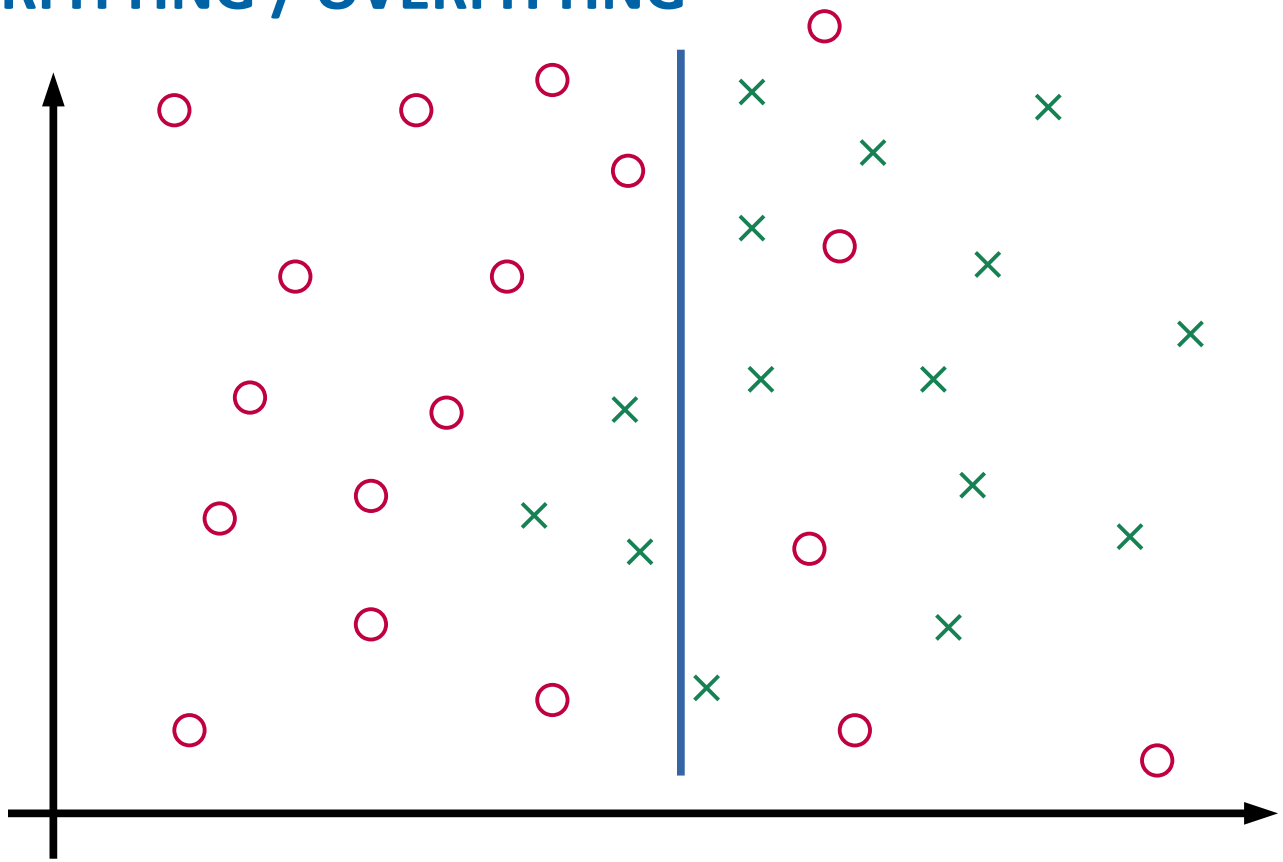
  - “When it has exactly 42042 oder 32059 leaves hat, it is a tree“

UNDERFITTING / OVERFITTING



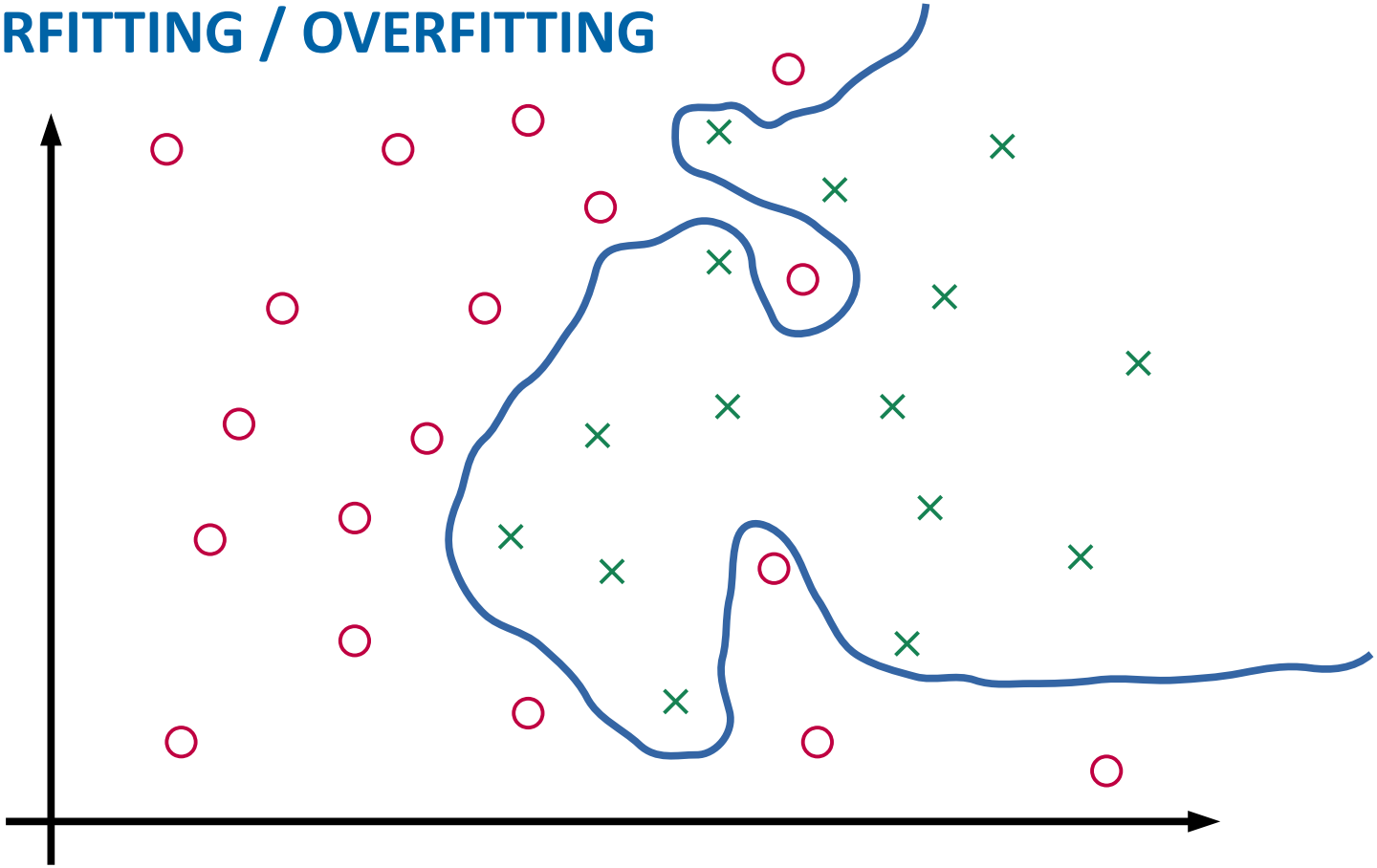
universität wien

UNDERFITTING / OVERFITTING



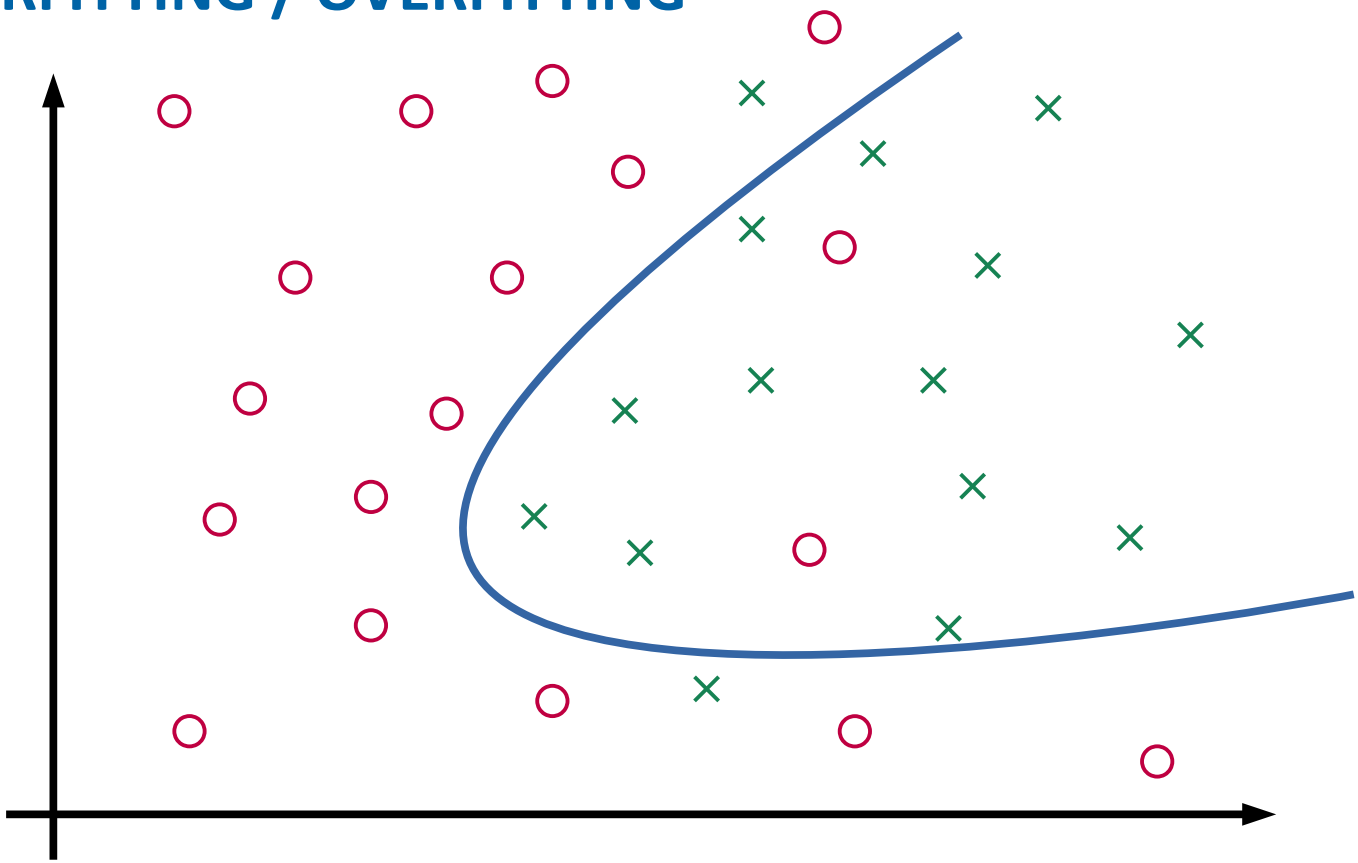
universität wien

UNDERFITTING / OVERFITTING



universität wien

UNDERFITTING / OVERFITTING



universität wien

FEATURE SELECTION

- Methods to reduce the number of features:
  - Compare feature vectors over all observations, e.g. with *Pearson correlation*

    Equal or “very similar“ dependent features don’t bring any gain and can be ignored

FEATURE SELECTION

- Methods to reduce the number of features:
  - Compare feature vectors over all observations, e.g. with *Pearson correlation*

    Equal or “very similar“ dependent features don’t bring any gain and can be ignored

  - Perform a PCA and use only the first *x* principal components

FEATURE SELECTION

- Methods to reduce the number of features:
  - Compare feature vectors over all observations, e.g. with *Pearson correlation*

    Equal or “very similar“ dependent features don’t bring any gain and can be ignored

  - Perform a PCA and use only the first *x* principal components

  - Compute coefficient of variance (CV) σ / μ for each feature vector

  - Features with too low CV behave equally upon all classes and can be ignored

APPLYING THE MODEL - PREDICTION

- As soon as a model is trained for a dataset, it can be “applied“
- All classification model a constructed, transform a feature vector into a nominal class

```
# Prediction of unknown iris flowers with the trained IRIS model

unknown = [[5.1, 3.5, 1.35, 0.2], [4.9, 3.5, 1.41, 0.2]]
# predict function works for all classifiers
result = model.predict(unknown)
print(result)
# ['setosa' 'setosa']
```

LEARNING STRATEGY

- How to learn correctly? Should I take all data to train the model?
- How to determine the goodness (prediction power) of a model?

LEARNING STRATEGY

- How to learn correctly? Should I take all data to train the model?
- How to determine the goodness (prediction power) of a model?
- Idea: split observations in dataset in training set and test set
- Train the model with the training set
- Make predictions upon the test set and compare predicted classes with true classes for each observation

VALIDATION

- Dataset needs to be split in a disjoint training set and test set

```
from sklearn.model_selection import train_test_split

# X: matrix with all numerichal features; y: list with all labels
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3,
random_state = 1)
```

- `test_size`: Size of the test set: 0.3 → 30 %
- `random_state`: determine of a start value for the internal random generator

VALIDATION

- After the training, apply the test set and compare with true labels
- We want the proportion of correctly classified labels from all classifications (accuracy):

```
model.fit(X_train, y_train)
# manual computation of the accuracy
predicted = np.array(model.predict(X_test), dtype = object)
accuracy = sum(predicted == Y_test) / len(Y_test)
```

VALIDATION

- After the training, apply the test set and compare with true labels
- We want the proportion of correctly classified labels from all classifications (accuracy):

```
model.fit(X_train, y_train)
# Computing the accuracy with the sklearn function
accuracy = model.score(X_test, y_test)
```

VALIDATION

- But if our dataset has two classes, the data is split 70 / 30 randomly and 69 / 70 and 1 / 30 observations are used for the training?

- Idea: Repeat validation with different random training sets and Testsets

CROSS VALIDATION

- Systematical method for testing a dataset
- Example: 4-fold cross validation

	Dataset			
Fold 1	Test	Train	Train	Train
Fold 2	Train	Test	Train	Train
Fold 3	Train	Train	Test	Train
Fold 4	Train	Train	Train	Test

CROSS VALIDATION

```
from sklearn.model_selection import cross_validate
model = tree.DecisionTreeClassifier(random_state = 0)

# computing of all accuracy values
cv_results = cross_validate(model, X, y, cv = 4)
print(cv_results['test_score'])
# array([0.97368421, 0.94736842, 0.94594595, 1.        ]

total_accuracy = cv_results['test_score'].mean()
print(total_accuracy)
# 0.9667496443812233
```

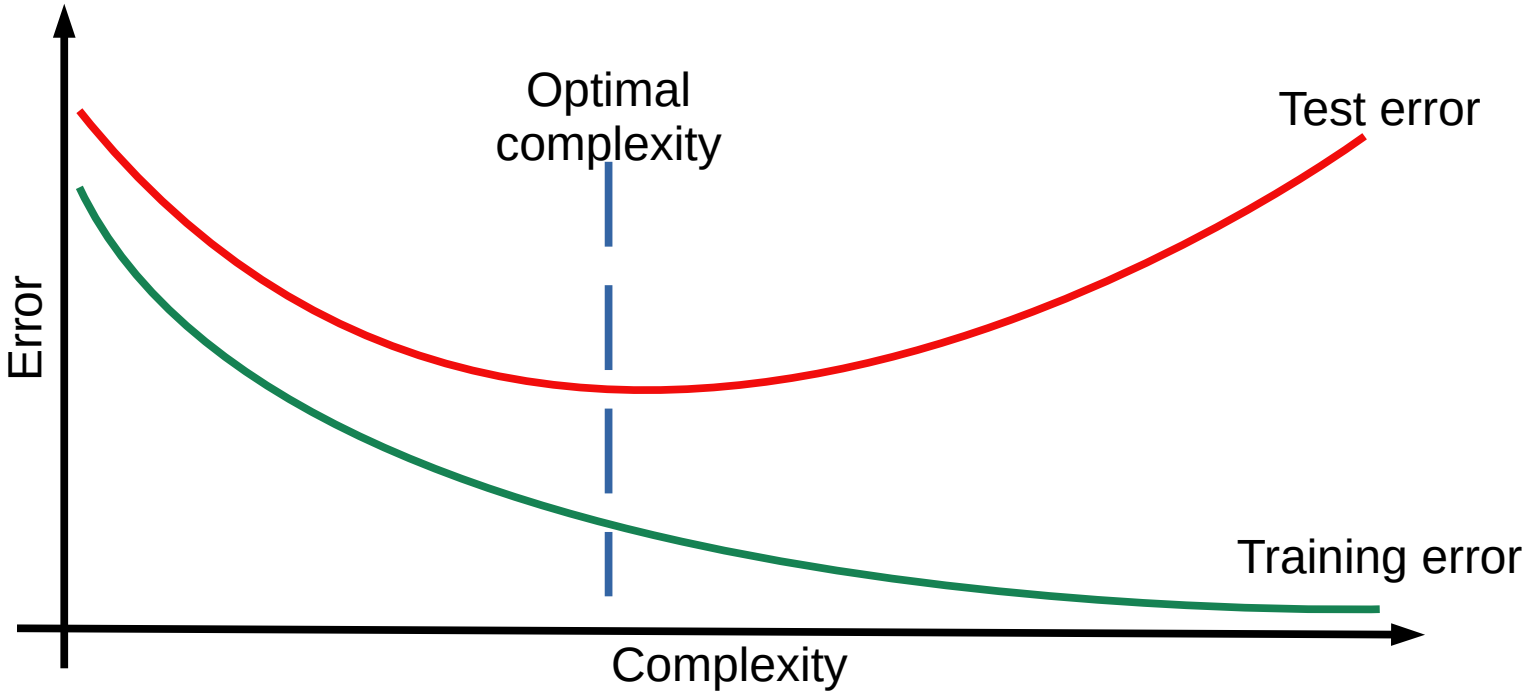
LEAVE ONE OUT

- Special case of cross validation, only one observation is taken for training

```
from sklearn.model_selection import cross_validate, LeaveOneOut
model, loo = tree.DecisionTreeClassifier(random_state = 0), LeaveOneOut()

# berechnen aller accuracy Werte auf einmal
cv_results = cross_validate(model, X, y, cv = loo.split(X))
print(cv_results['test_score'])
# [1. 1. 0. 1. 1. 1. 1. ... 1. 1. 1. 0. 1.]

total_accuracy = cv_results['test_score'].mean()
print(total_accuracy)
# 0.953333333333333
```

OPTIMAL MODEL WITH PARAMETER TUNING

- Determine complexity of a model with parameter changing (tuning)




```
test_error = 1 - model.score(X_test, y_test)
training_error = 1 - model.score(X_training, y_training)
```

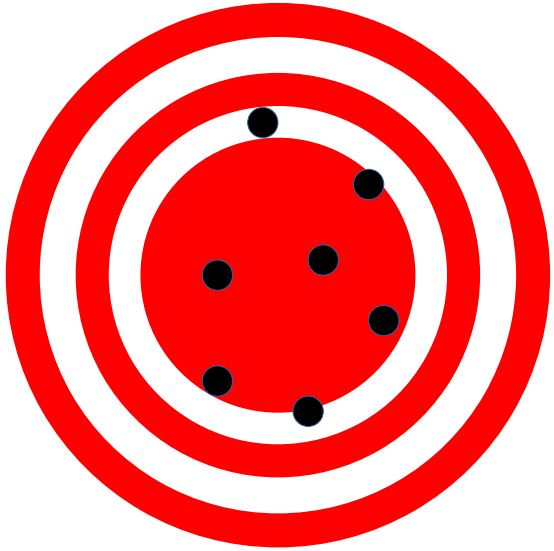
universität wien

GOODNESS FOR BINARY CLASSIFICATION

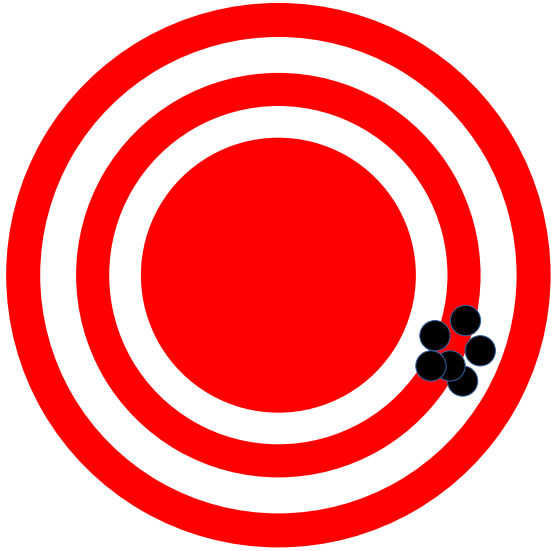
		Predicted condition	
		positive	negative
True condition	positive	True positive (TP)	False negative (FN, Type 2 error)
	negative	False positive (FP, Type 1 error)	True negative (TN)

- Sensitivity: TP / (TP + FP)
- Specificity: TN / (TN + FN)
- Precsion: TP / (TP + FP)
- False discovery rate: FP / (FP + TP)
- Accuracy: (TP + TN) / (TP + FP + FN + TN)

GOODNESS FOR BINARY CLASSIFICATION



High sensitivity
low precision



Low sensitivity
high precision

universität
wien

EXECUTION OF A CLASSIFICATION EXPERIMENT

1) Load dataset

2) Choose a model

3) Determine with x-fold cross validation the correlation (total accuracy) between the features and the target variable

4) Train a model on complete dataset

5) Predict unlabelled observations with complete model